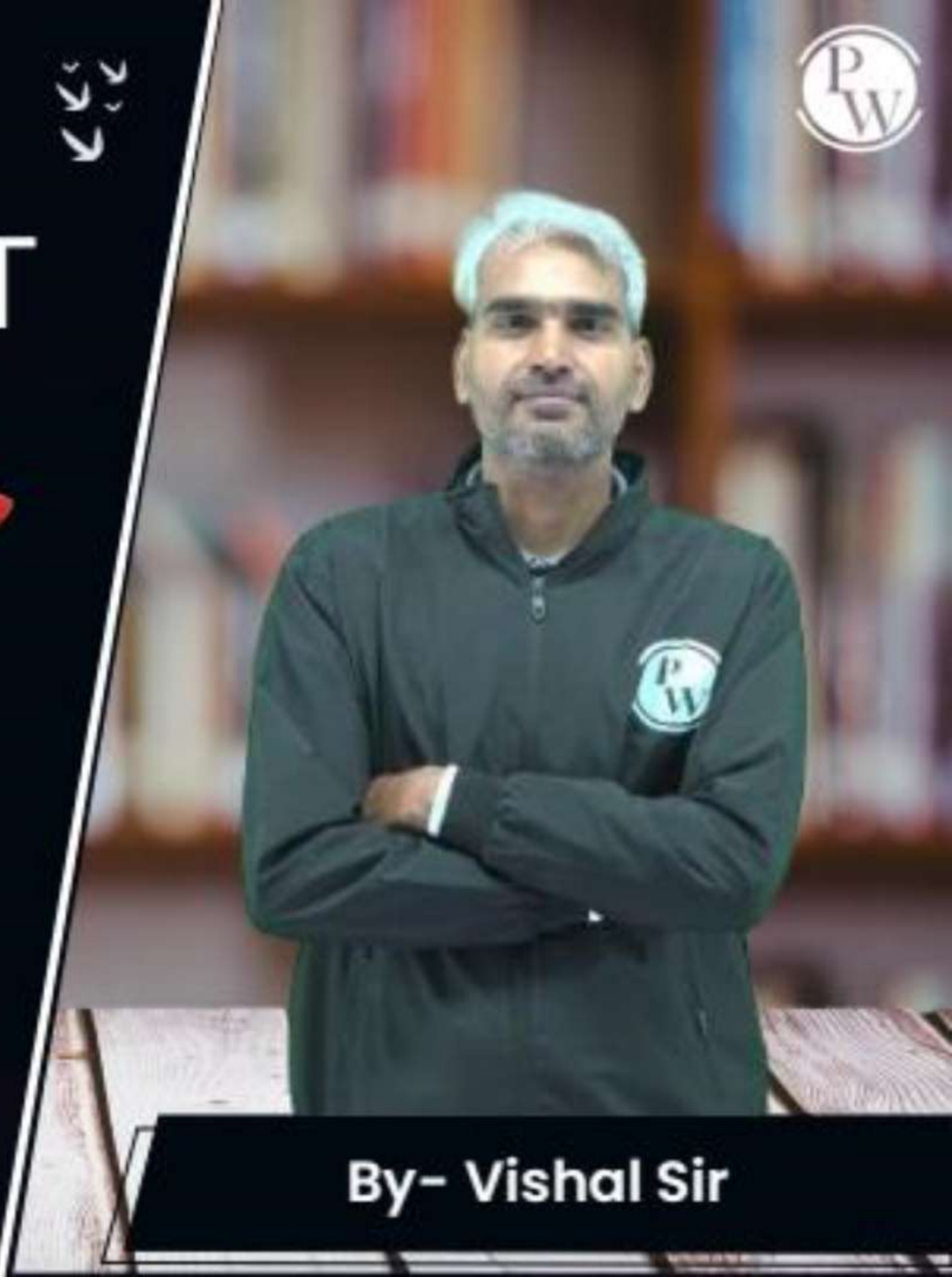
Computer Science & IT



Compiler Design

Introduction and Lexical Analysis

Lecture No. 05



By- Vishal Sir

Recap of Previous Lecture



Topic Process of translation

Topic Basic concept of lexical analyzer

Slide

Topics to be Covered



- Topic: Tokens in C
- Topic: Questions

Slide



Topic : Quick Revision

Functionality of Lexical Analyzer :

1. It groups stream of characters into tokens.
2. It generates lexical errors (if any).
3. It eliminates(discards) white spaces and comments.
4. It starts the construction of symbol table.
5. It maintains line numbers.
6. It follows longest prefix Rule



Topic : Types of tokens in C

C language is Case sensitive

1. Keywords in C
2. Identifiers in C
3. Operators in C
4. Special Symbols in C
5. Constants in C
6. String in C

Slide

{All Keywords are lower case}

Keywords in C : Keywords are pre-defined words in C-language {Can not be used as an identifier}

↓

i.e. Variable name

(or)

User defined function

* There are 32 Keywords in Standard C.
* Basic eg: int, float, if, else, while etc.

Important eg:- return, register, sizeof

Read about all 32 Keywords

Slide

Identifiers in C : Identifiers are user defined words used for identifying the variable or user defined function

Rules to define identifier

① Letters, digit and underscores can be used to define an identifier

- Letter → (a-z), (A-Z)
- digit → (0-9)
- underscore → _

Slide

② First character of an identifier must be letter or underscore, and after that it can be any combination of letter, digit and underscore.

{a, b, ab,
_a, _a_b, _1,
_a1,

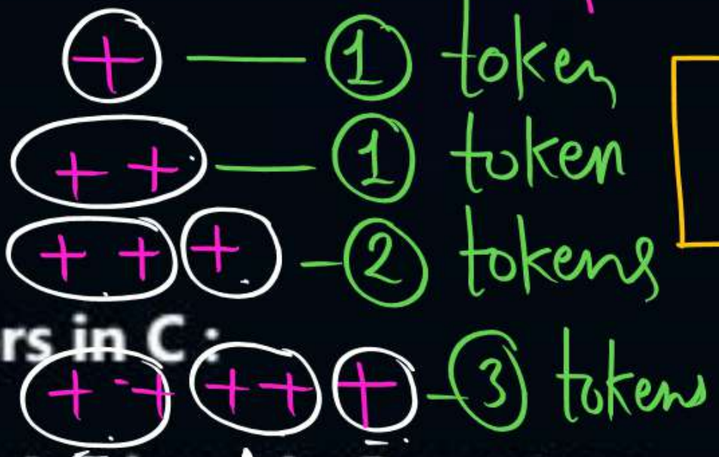
{a1, a12b,

{1a}

③ It can not be a keyword

④ It is case sensitive

⑤ No white space.



$x+=3 \Rightarrow x=x+3$

$=+$ Not an assignment op^{n}

Operators in C :

Arithmetic Operators

$+$

$-$

$*$

$/$

$\%$

Unary $\{ ++ \;\; -- \}$

Assignment Operators

$=$ $\quad$ $\&=$ $\quad$ $\&\&=$

$+=$ $\quad$ $|=$ $\quad$ $||=$

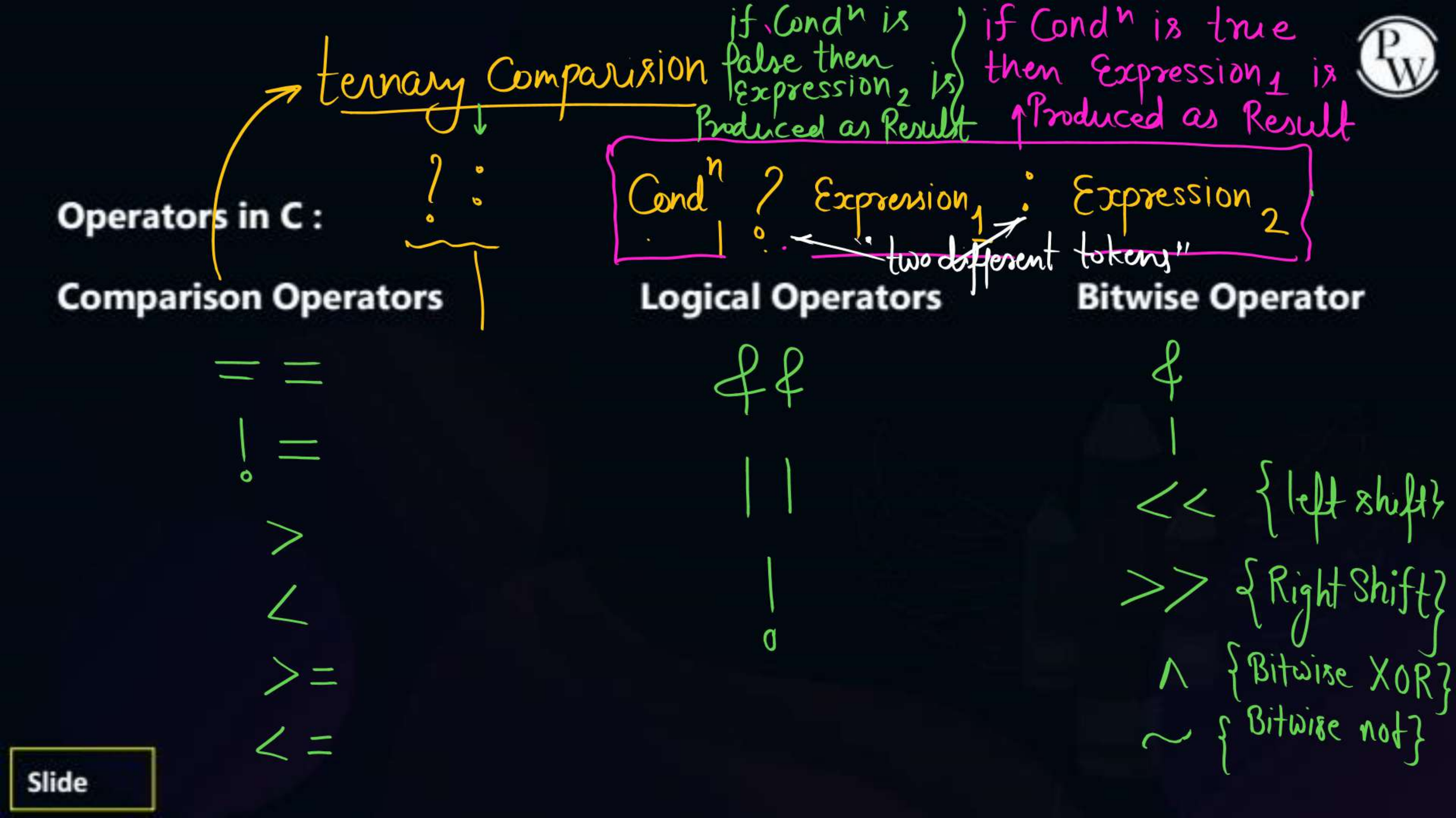
$*=$ $\quad$ $\wedge=$

$-=$ $\quad$ $<<=$

$/=$ $\quad$ $>>=$

$\%=$

Slide


ternary Comparision
if Cond^n is false then Expression_2 is Produced as Result
if Cond^n is true then Expression_1 is Produced as Result
Cond^n ? Expression_1 : Expression_2
two different tokens
Operators in C :
Comparison Operators
Logical Operators
Bitwise Operator
==
!=
>
<
>=
<=
&&
||
!
&
<< {left shift}
>> {Right Shift}
^ {Bitwise XOR}
~ {Bitwise not}
Slide

Special Symbols in C :

eg:

(	*
)	?
{	:
}	;
[	,
]	etc.

Slide

Constants in C :

Constant may be,

① Integer or decimal Constants

② Octal Constants { Starts with 0 and possible decimal digits after 0 ranges between 0 & 7 }

③ Hexadecimal Constants. { Starts with 0x / 0X and remaining numbers are in Hexadecimal range }

④ Character Constants

eg: 'a' 'b' etc

⑤ String Constants

eg: "abc"

"Single String Constant"

Slide

Same as String Constants

Strings in C : String is a array of characters

eg Char a[ ] = "Compiler design"

String or String Constant

Slide



Topic : Lexical Analysis

int Keyword max identifier (Sp. Sym

#Q. Consider the following program segment.

```
int max(i,j)
{
    int i,j;
    /* function returns
                maximum of i and j */
    return  i>j?i:j;
}
```

i id , Sp. Sym

A. Lexical Error

B. No Lexical Error and Number of tokens created = 22

C. No Lexical Error and Number of tokens created = 23

D) None

Slide



Topic : Lexical Analysis

#Q. Consider the following program segment.

```
int a, ab ;
```

A. Lexical Error

B. No Lexical Error and Number of tokens created = 5

C. No Lexical Error and Number of tokens created = 6

D. None of the above

Slide



Topic : Lexical Analysis

#Q. Consider the following program segment.

```
int a ab ;
```

A. Lexical Error

B. No Lexical Error and Number of tokens created = 4

C. No Lexical Error and Number of tokens created = 5

D. None of the above



Topic : Lexical Analysis

#Q. Consider the following program segment.

```
int a , ab , ab1 , 12 , ab12cd  ;
```

A. Lexical Error

B. No Lexical Error and Number of tokens created = 11

C. No Lexical Error and Number of tokens created = 13

D. None of the above



Topic : Lexical Analysis

#Q. Consider the following program segment.

```
int a , ab , ab1 , 12 , ab12cd  ;
```

A. Lexical Error

B. No Lexical Error and Number of tokens created = 11

C. No Lexical Error and Number of tokens created = 13

D. None of the above



Topic : Lexical Analysis

#Q. Consider the following program segment.

```
int a, ab , abc ab12 , _ab , a_bc  a123, 23 , 23 a ;
```

A. Lexical Error

B. No Lexical Error and Number of tokens created = 15

C. No Lexical Error and Number of tokens created = 16

D. None of the above

Slide



Topic : Lexical Analysis

#Q. Consider the following program segment.

```
int a, ab , 23a ;
```

A. Lexical Error

B. No Lexical Error and Number of tokens created = 8

C. No Lexical Error and Number of tokens created = 7

D. None of the above



Topic : Lexical Analysis

#Q. Consider the following program segment.

```
printf ( "Hello students ans = %d", i );
```

Single token

A. Lexical Error

B. No Lexical Error and Number of tokens created = 14

C. No Lexical Error and Number of tokens created = 7

D. None of the above

Slide



Topic : Lexical Analysis

#Q. Consider the following program segment.

```
printf ( "i = %d , &i  = %x", i  , &i );
```

A. Lexical Error

B. No Lexical Error and Number of tokens created = 10

C. No Lexical Error and Number of tokens created = 9

D. None of the above



Topic : Lexical Analysis

#Q. Consider the following program segment.

```
main ()
{
        fro(i=0 ; i <=10 ; i++)
}
```

A. Lexical Error

B. No Lexical Error and Number of tokens created = 18

C. No Lexical Error and Number of tokens created = 19

D. None of the above



2 mins Summary

Topic Tokens in C

Topic Identification of lexical error (if any), and number of tokens

Slide

THANK - YOU